MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Command Line and API Management

Unit 11 · Virtualization and SDDC (10-150-130)

MID-STATE TECHNICAL COLLEGE

Learning Outcomes

By the end of this unit, you will be able to:



Manage Proxmox VE from the shell

Inspect, control, and back up VMs and containers with `qm`, `pct`, `vzdump`, and `pvesm`.



Automate routine tasks with scripts

Write Bash scripts that loop over guests, check for errors, log results, and run on a schedule.



Call the Proxmox REST API

Authenticate with an API token and query or change resources with `curl`, `pvesh`, and Python.



Explain infrastructure as code

Describe how Terraform, Ansible, Git, and CI/CD pipelines turn infrastructure into reviewed, repeatable code.

LESSON 1

The Proxmox VE Command Line



Why Manage from the Command Line?

- **Speed and precision** — one command replaces a dozen clicks, and the exact syntax documents what was done.
- **Scriptability** — commands chain into scripts that run the same way on every node, every time.
- **Low bandwidth** — an SSH session works over a slow VPN link where the web UI struggles.
- **Last line of access** — when the web interface or `pveproxy` is down, the shell still works.
- **Full feature reach** — some options (disk import, advanced flags) are CLI-only.



What you are really on

Proxmox VE 9.x runs on **Debian 13 (Trixie)**. The shell is a normal Debian root shell — `apt`, `systemctl`, and `journalctl` all apply.

Getting to the Shell

Two common ways in — both land you in a root shell on the node:

```
ssh root@WRPOD01-Host1.mstclab.local          # or ssh root@10.22.3.yy
pveversion -v                                 # confirm release and package versions
hostname; ip -br addr                         # which node, which addresses
```

Web UI Shell button

- Node → **>_ Shell** opens a terminal in the browser
- No SSH client needed; uses the same login session

SSH

- Standard `ssh` from any workstation
- What scripts, Ansible, and remote admins use

LIVE DEMO *SSH to Host1 by name, then by IP; run pveversion -v.*

The Proxmox VE Toolset

Each Proxmox subsystem has its own command — learn the prefix and the help is built in:

Tool	Manages	Example
`qm`	QEMU/KVM virtual machines	`qm list`
`pct`	LXC containers	`pct exec 101 -- uptime`
`vzdump`	Backups of VMs and containers	`vzdump 100 --storage local`
`pvesm`	Storage definitions and content	`pvesm status`
`pveum`	Users, groups, roles, and API tokens	`pveum user list`
`pvesh`	The whole REST API from the shell	`pvesh get /nodes`
`pvecm`	Cluster membership and quorum	`pvecm status`

Add `help` or open `man qm` for every subcommand and flag.

Managing VMs with qm

```
qm list                          # VMID, name, status
qm config 100                    # full VM definition
qm start 100
qm shutdown 100                  # graceful (ACPI)
qm stop 100                      # hard power-off
qm set 100 --memory 4096 --cores 2
qm snapshot 100 pre-update
qm clone 9000 120 --name web01 --full
```



Where the config lives

Each VM is a text file: `/etc/pve/qemu-server/100.conf`. `qm config` reads it; `qm set` edits it safely.

Shutdown vs. stop: use `qm shutdown` for a clean stop; `qm stop` is the equivalent of pulling the plug.

LIVE DEMO *List VMs, read a config, change memory with qm set, confirm in the GUI.*

Managing Containers with pct

Same verbs as qm

- `pct list` / `pct config 101`
- `pct start 101` / `pct shutdown 101`
- `pct set 101 --memory 1024`
- `pct snapshot 101 before-patch`

Container-only

- `pct enter 101` — open a shell inside the container
- `pct exec 101 -- apt update` — run one command inside
- `pct push` / `pct pull` — copy files in and out

Why exec works without a login

A container shares the host kernel, so the node can step straight into its namespaces. A VM has its own kernel — you would need SSH or the guest agent instead.

Backup and Restore with vzdump

One backup tool for both guest types — the **mode** controls the downtime trade-off:

Mode	What happens	Downtime
`snapshot`	Live backup; consistency via guest agent / storage snapshot	None (default)
`suspend`	Guest paused while data is copied	Short pause
`stop`	Guest shut down, backed up, restarted	Full outage, most consistent

```
vzdump 100 --storage local --mode snapshot --compress zstd
qmrestore /var/lib/vz/dump/vzdump-qemu-100-<date>.vma.zst 130     # VM: file, then new VMID
pct restore 131 /var/lib/vz/dump/vzdump-lxc-101-<date>.tar.zst    # CT: new VMID, then file
```

Storage and Node Health Commands

Proxmox view

- `pvesm status` — every storage, type, and free space
- `pvesm list Local-Disk` — what lives on one store
- `pveperf` — quick CPU, fsync, and DNS check

Underlying Linux view

- `lvs` / `vgs` / `pvs` — LVM volumes, groups, disks
- `lsblk` and `df -h` — block devices and filesystems
- `iostat -x 2` — live disk latency (`sysstat`)

Read before you write: listing commands are safe, but `lvremove`, `vgreduce`, and `wipefs` destroy data instantly. Confirm the VMID and device twice.

LIVE DEMO *Compare pvesm status with lvs on the same node.*

LESSON 2

CLI Scripting and Automation

Anatomy of a Safe Admin Script

```
#!/usr/bin/env bash
set -euo pipefail                  # stop on errors, unset vars
LOG_TAG="lab-backup"
log() { logger -t "$LOG_TAG" "$*"; echo "$*"; }

VMID="${1:?usage: $0 <vmid>}"      # require an argument
qm status "$VMID" >/dev/null       # fail fast if missing
log "Backing up $VMID"
vzdump "$VMID" --storage local --mode snapshot
log "Backup of $VMID complete"
```

Habits that pay off

`set -euo pipefail` — never run past a failure

Validate inputs before acting

`logger` — results land in `journalctl -t lab-backup`

One job per script; small and testable

Looping Over Guests

Turn a one-off command into a fleet operation:

```
# Snapshot every running VM before patching
for vmid in $(qm list | awk 'NR>1 && $3=="running" {print $1}'); do
  qm snapshot "$vmid" "pre-patch-$(date +%Y%m%d)" && echo "OK  $vmid" || echo "FAIL $vmid"
done

# More robust: ask the API for JSON and filter with jq
pvesh get /nodes/$(hostname)/qemu --output-format json \
  | jq -r '.[] | select(.status=="running") | .vmid'
```

Text columns can change between releases; JSON fields are a stable contract. Prefer JSON in anything you keep.

Scheduling Automation

Proxmox backup jobs

- Datacenter → **Backup** → Add
- Stored cluster-wide in `/etc/pve/jobs.cfg`
- Retention, notifications, and visibility built in
- Use for all routine backups

cron / systemd timers

- `crontab -e` → `0 2 * * * /root/bin/snap-all.sh`
- Runs on one node only
- Check results in `journalctl`
- Use for custom scripts and reports

Rule of thumb: if Proxmox has a native scheduler for the task, use it — the next administrator will find it in the GUI.

Scripts Run with Real Power

- Node scripts run as **root** — a bad loop can stop or delete every guest at once.
- **Dry run first**: prefix the action with `echo` and read the output before removing it.
- Test against one **throwaway VMID** before pointing the script at production guests.
- Keep scripts in **Git** — never edit the only copy live on the node.



SECURITY FOCUS

Never hard-code passwords or token secrets in a script. Read them from a root-only file (chmod 600) or an environment variable.

Why it matters: Scripts get copied, emailed, and committed — and the secret goes with them.

LESSON 3

Management Using the REST API



REST in Five Ideas

REST (Representational State Transfer) is an architectural style for web APIs:

Idea	What it means in practice
Resources	Everything has a URL path — a node, a VM, a storage: `/nodes/host1/qemu/100`
HTTP methods	The verb is the action — `GET` reads, `POST` creates, `PUT` updates, `DELETE` removes
Representations	Data travels as JSON, readable by people and every programming language
Stateless	Each request carries its own credentials; the server keeps no session memory
Status codes	The HTTP code says how it went — `200` OK, `401` not authenticated, `403` not permitted

Proxmox calls its API "REST-like": it follows these principles and is fully described in the online API viewer.

How the Proxmox API Is Organized

Every call starts at `https://<node>:8006/api2/json` and walks a tree of resources:

```
GET    /nodes                                  # list cluster nodes
GET    /nodes/{node}/qemu                      # list VMs on a node
POST   /nodes/{node}/qemu                      # create a VM
GET    /nodes/{node}/qemu/{vmid}/config        # read VM config
PUT    /nodes/{node}/qemu/{vmid}/config        # change VM config
POST   /nodes/{node}/qemu/{vmid}/status/start  # power actions
DELETE /nodes/{node}/qemu/{vmid}               # destroy a VM
```

The GUI is an API client

Open your browser's developer tools on the Proxmox UI and watch the Network tab — every click is one of these calls.

Authenticating to the API

Proxmox VE accepts two kinds of credentials:

	Ticket (cookie)	API token
How you get it	POST user + password to `/access/ticket`	Create once: GUI or `pveum user token add`
How it is sent	Cookie `PVEAuthCookie` + `CSRFPreventionToken` on writes	Header `Authorization: PVEAPIToken=…`
Lifetime	Expires after 2 hours	Until its expiry date or revoked
Best for	Interactive use — the web UI	Scripts, Terraform, Ansible, monitoring

TLS on port 8006 encrypts every request; it does not replace a ticket or token.

API Tokens and Privilege Separation

- A token belongs to a user: `automation@pve!lab` = user `automation@pve`, token ID `lab`.
- The **secret** (a UUID) is displayed **once** at creation — store it immediately.
- **Privilege separation** (`--privsep 1`, the default) gives the token its own, smaller set of permissions.
- Tokens can carry an **expiry date** and be revoked without disabling the user.

```
pveum user token add automation@pve lab --privsep 1
pveum acl modify /vms --tokens 'automation@pve!lab' --roles
PVEAuditor
```



SECURITY FOCUS

Give each script or tool its own token with only the role it needs. Never automate as root@pam.

Why it matters: A leaked root password owns the whole cluster; a leaked read-only token can only look.

Calling the API with curl

```
export TOKEN="PVEAPIToken=automation@pve!lab=<secret-uuid>"
export PVE="https://WRPOD01-Host1.mstclab.local:8006/api2/json"

curl -sk -H "Authorization: $TOKEN" "$PVE/nodes" | jq
curl -sk -H "Authorization: $TOKEN" "$PVE/nodes/{node}/qemu" | jq '.data[] | {vmid, name, status}'
curl -sk -X POST -H "Authorization: $TOKEN" "$PVE/nodes/{node}/qemu/100/status/start"
```

About -k: it turns off certificate checking. Fine against our self-signed lab hosts; in production, trust the CA instead so a spoofed host cannot capture your token.

Status Codes and Asynchronous Tasks

Code	Meaning	Usual cause
`200`	Success	Data returned, or a task was queued
`400`	Parameter verification failed	Missing or invalid field — read the error body
`401`	Not authenticated	Bad, expired, or malformed token header
`403`	Permission check failed	Token lacks the privilege on that path
`500`	Server-side error	Operation failed — check the task log

Long jobs return a task ID

Start, clone, and backup return a `UPID` at once. Poll `/nodes/{node}/tasks/{upid}/status` until it reports `stopped` with `exitstatus: OK`.

pvesh — The API from the Shell

```
pvesh ls /nodes                    # browse the tree
pvesh get /version
pvesh get /cluster/resources --type vm
pvesh usage /nodes/{node}/qemu -v   # show parameters
pvesh create /nodes/{node}/qemu/100/status/start
pvesh set /nodes/{node}/qemu/100/config --memory 4096
```

pvesh verb	HTTP method	Action
get	GET	Read
create	POST	Create / act
set	PUT	Update



Why learn it

Runs as root on the node — no token needed — and proxies to other cluster members. Whatever works in `pvesh` works in `curl` with the same path.

Scripting the API in Python

`proxmoxer` maps API paths onto Python calls — install with `pip install proxmoxer requests`:

```
import os
from proxmoxer import ProxmoxAPI

pve = ProxmoxAPI("WRPOD01-Host1.mstclab.local", user="automation@pve",
                 token_name="lab", token_value=os.environ["PVE_TOKEN"],
                 verify_ssl=False)          # lab only — self-signed certificate

for vm in pve.nodes("WRPOD01-Host1").qemu.get():     # GET /nodes/{node}/qemu
    print(vm["vmid"], vm["name"], vm["status"])
```

The same pattern feeds dashboards, inventory reports, and chat-ops bots — anything that needs live data from the cluster.

LESSON 4

Infrastructure as Code



Imperative vs. Declarative

Imperative — "how"

- A sequence of steps: create, then set, then start
- Running it twice may do the work twice
- Examples: Bash scripts, `curl` calls, `pvesh`

Declarative — "what"

- Describe the desired end state: "VM web01 with 2 cores"
- The tool compares desired and actual, then **changes only the difference**
- Examples: Terraform, Ansible modules, Kubernetes manifests

Idempotent: applying the same code again produces the same result and no extra changes. It is what makes IaC safe to re-run.

The IaC Toolchain for Proxmox VE

Category	Tools	Proxmox integration	Best for
Provisioning	Terraform, OpenTofu, Pulumi	`bpg/proxmox` provider	Creating VMs, CTs, networks
Configuration management	Ansible, Puppet, Chef	`community.proxmox` collection	Guest OS and software setup
Image building	Packer	`proxmox-iso` builder	Repeatable VM templates
Container orchestration	Kubernetes, Docker Swarm	Runs on VMs Proxmox hosts	Scalable applications
GitOps	Argo CD, Flux	Git repo is the source of truth	Continuous deployment

All Proxmox integrations here are community-maintained — pin provider and collection versions in your code.

Terraform: Declare a VM

```
provider "proxmox" {
  endpoint = "https://WRPOD01-Host1.mstclab.local:8006/"
  insecure = true     # lab self-signed cert
}                     # token via PROXMOX_VE_API_TOKEN

resource "proxmox_virtual_environment_vm" "web01" {
  name      = "web01"
  node_name = "WRPOD01-Host1"
  clone  { vm_id = 9000 }
  cpu    { cores = 2 }
  memory { dedicated = 2048 }
}
```

The workflow

`terraform init` — download the provider

`terraform plan` — preview the changes

`terraform apply` — make reality match

`terraform destroy` — tear it all down

State in `terraform.tfstate` records what was built.

Ansible: Configure with Playbooks

```
- name: Clone web VM from template
  hosts: localhost
  tasks:
    - community.proxmox.proxmox_kvm:
        api_host: WRPOD01-Host1.mstclab.local
        api_user: automation@pve
        api_token_id: ansible
        api_token_secret: "{{ vault_pve_token }}"
        node: WRPOD01-Host1
        clone: ubuntu-tmpl
        vmid: 9000
        newid: 120
        name: web01
        state: present
```

- **Agentless** — SSH to guests, or the API for Proxmox itself
- Playbooks are **YAML** — readable by non-programmers
- Secrets live in `ansible-vault`, not in the playbook
- Pairs with Terraform: **build**, then **configure**

Git and CI/CD for Infrastructure

Treat infrastructure changes like software changes — every change reviewed, tested, and recorded:



Commit and review

Changes go on a branch and through a pull request; a teammate approves before merge.



Validate automatically

The pipeline runs `terraform validate`, `terraform plan`, and `ansible-lint` on every commit.



Deploy in stages

Apply to a test environment first, then production — with the plan output as the record.



Audit and roll back

Git history shows who changed what and why; reverting a commit restores the previous state.

Never commit `terraform.tfstate`, `*.tfvars` with secrets, or token values — add them to `.gitignore`.

Security Across the Unit



SECURITY FOCUS

Least privilege: one user or token per script and tool, with privilege separation and only the role it needs — never root@pam for automation.

Protect secrets: tokens and passwords live in root-only files, environment variables, or a vault — never in scripts, playbooks, or Git.

Verify connections: `curl -k` and `insecure = true` are lab shortcuts; production trusts a real CA.

Review and log: scripts dry-run first, IaC changes go through a pull request, and every action leaves a log entry.

Why it matters: The CLI, API, and IaC tools can each change every guest at once — so can an attacker holding their credentials.

Hands-On Lab: Command Line Interface

NETLAB+ · Lab 13 – Command Line Interface (CLI) · your assigned POD hosts



- Identify VMs and containers with `qm list` and `pct list`.
- Read and change guest configuration with `qm config` and `qm set`.
- Create a backup with `vzdump` and confirm it in storage.
- Inspect storage with `pvesm`, `lvs`, `vgs`, and `pvs`.
- Measure node performance with `pveperf` and record the results.

See Blackboard Ultra for the lab assignment and submission details.

Key Takeaways

- The shell is a full management interface — `qm`, `pct`, `vzdump`, and `pvesm` cover daily operations.
- Good scripts fail fast, log results, and dry-run before acting: `set -euo pipefail`.
- Everything in the GUI is a REST call to `:8006/api2/json` — `pvesh` and `curl` reach the same API.
- Automate with API tokens and privilege separation — never with the root password.
- Infrastructure as code is declarative and idempotent, versioned in Git, and deployed through review.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 11 · Command Line and API Management

MID-STATE TECHNICAL COLLEGE